

Predicting FEMA Disaster-Assistance Eligibility: Hurricane Ian, Lee County, FL

CAI 4105 Machine Learning, Fall 2026, Project Milestone 1: Initial Report

Team FEMA | Anthony Stoneking, Daniel Ortiz, Nandor Laar | Due **Wednesday, October 7, 2026**

1. Executive Summary

Our project is trying to predict whether FEMA gives a household aid after Hurricane Ian, specifically in Lee County, FL. The data that we're using is directly from FEMA, the 194,482 applications and 100 columns, with a near 50/50 split of those who received aid and those who didn't.

One of the most important things we found was that 43 out of the 100 columns are results of FEMA's decision, leakage, so we can't use them. We separate it into two tiers. Tier 1: 25 features known at registration, and Tier 2: 39 features that are known after inspection.

The plan is to do a 60% train, 20% validation, and 20% test split, try four models to see how it changes, do a 5-fold cross-validation, and score with ROC-AUC. We cross-validate so that we can see how much the score changes from run to run, so that we aren't basing our model off of one run. What's next is to build and test models for Milestone 2.

2. Business Understanding

2.1 Problem definition and motivation

Our analysis

After a big hurricane like Ian back in 2022, tens of thousands of people and households applied for federal aid within days of the disaster. In Lee county, applications peaked September 30th, 2022 (15,240 in one day), and 94% arrived by the end of October (Figure 2.1). Each application goes through a referral, documentation checks, with sometimes an inspection and a decision. Around half of the people end without an award and for various reasons: 30% of those had insurance, 19% had no eligible damage or needs and 13% did not respond or just withdrew from the application, and 35% were never referred to the program (Figure 2.2).

The question our model answers: *Given what we know about an applicant, will FEMA award them IHP aid?*

Why it matters. Applicants wait for answers while caseworkers are overloaded. A reliable prediction could help FEMA:

- (a) route likely eligible applications to fast track;
- (b) flag applications that are likely ineligible early, so staff can tell people what is missing (for example, insurance documents);
- (c) decide where to send inspectors first;
- (d) plan staffing and budget for the next disaster.

What it is not. The model is decision *support*. It must never automatically deny anyone aid. A person makes every final decision.

AI-assisted analysis



Figure 2.1. Daily applications; the peak is Sep 30, 2022.

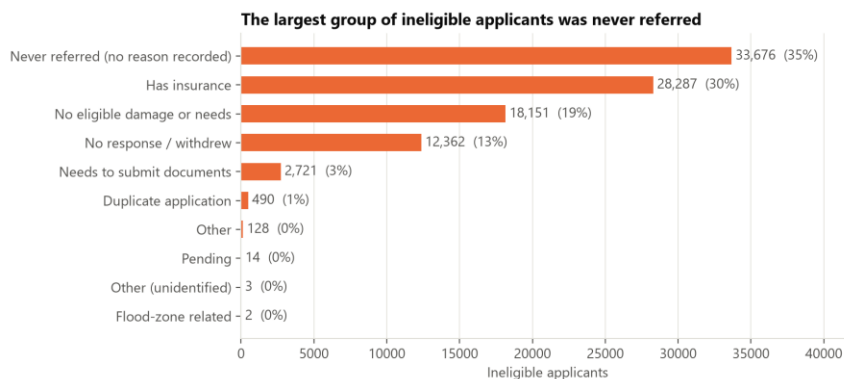


Figure 2.2. Why applicants were not awarded aid.

2.2 Business objectives and success criteria

AI-assisted analysis

Objective	How we will measure it	Proposed target
Predict if someone gets aid right when they apply (Tier 1)	ROC-AUC and F1 on the held-out test set	Beat the untuned baseline: ROC-AUC ≥ 0.90 , F1 ≥ 0.91
Predict if someone gets aid after the inspection (Tier 2)	Same as above	ROC-AUC ≥ 0.94
Do better than the simple approaches	Compare against just guessing the majority class and against logistic regression	Clearly beat both (guessing the majority class gets 50.7% accuracy; the logistic-regression

		reference is re-run on the verified data, see Section 5.1)
Be fair to every group	Error rates for each age band and income band	No group's error rate is more than 5 percentage points above the overall rate
Be explainable	Feature-importance and partial-dependence plots that a caseworker could actually read	The main drivers line up with FEMA's own eligibility rules

Note. These targets were set from earlier untuned baselines that must be re-run on the verified data (Section 5.1) before they are confirmed. They are goals for the tuned models, not promises.

2.3 Stakeholder analysis

AI-assisted analysis

Stakeholder	Interest	Cost of a wrong prediction
Applicants (survivors)	Quick and fair decisions that they can understand	A household wrongly flagged as "ineligible" (a false negative) might just give up on a claim they deserve
FEMA caseworkers and program managers	Getting through applications fast and accurately, with decisions they can defend	Wasted inspections (false positives), or eligible people getting missed
Lee County emergency management, State of Florida	Faster recovery and being able to plan their resources	Misjudging how much help is needed
Congress, auditors, taxpayers	Federal money being used properly	Improper payments or denials that nobody can explain
Equity and legal-aid organizations	No group being treated worse than another	Bias against certain ages, income levels, or places
Our team / instructor	A solid method and honest reporting	n/a

2.4 Expected impact and value proposition

Our analysis

A tuned, audited model would give FEMA an explainable signal at registration that also comes in early, when information is cheapest to act on. Its value is speed and prioritization, not replacing judgment. Because the outcome depends partially on FEMA's own screening rules, the model also works as a *consistency check*: applications where the model strongly disagrees with the outcome deserve a second check.

2.5 Risks and assumptions

Our analysis

- **Selection bias:** only *valid* registrations are in the data, so we can say nothing about invalid applications.

- **Scope / generalization:** one storm and one county, so results may not transfer to other disasters.
- **Label noise:** "not eligible" mixes many reasons (insurance, missing documents, withdrawal, never referred).
- **Fairness:** age, income, and location can correlate with vulnerable groups, so we will audit error rates by group.

AI-assisted analysis

- The flat \$700 payment (confirmed): the \$700 award is FEMA's *Serious Needs Assistance*, a one time and flexible payment per household for urgent needs: food, water, and medication, approved soon after registration (FEMA, *Serious Needs Assistance* fact sheet). Because it can be approved early and follows a simple rule, part of what our model learns is this screening part.

3. Data Understanding

Each subsection has two parts so it's clear which analysis is ours and which was AI-assisted: **Our analysis** (written by me) and **AI-assisted analysis** (written with Claude from my answers). Some sentences in "Our analysis" started as AI drafts that I rewrote in my own words; the AI log (docs/AI_USAGE.md) lists them. Numbers come from notebooks/eda/01_initial_eda.ipynb.

3.1 Dataset description and source

Our analysis

Our data comes from FEMA's *Individuals and Households Program – Valid Registrations (v2)*, a public government dataset drawn from FEMA's National Emergency Management Information System (NEMIS). We downloaded the data from FEMA's public data site using our script `src/data/fetch_fema.py`, keeping it to Hurricane Ian (DR-4673) and to applicants in Lee County. The data was pulled with a Python script and then looked at and summarized. An AI tool wrote the first pass of the analysis. We then re-ran it on the verified data and checked every number against the data file. Because FEMA refreshes the data weekly, the row count can drift slightly over time. The size of the data is 194,482 rows by 100 columns, with applications dated 2022-09-27 to 2023-01-12. One row is one household's application, which is our unit of observation. FEMA warns that this is raw data that has human error. It includes only valid registrations.

ML takeaway: The dataset is large enough for k-fold cross-validation without starving any fold, but because it covers a single disaster, it limits generalization.

AI-assisted analysis

None for this part beyond grammar fixes and checking the row and column counts against the data file.

3.2 The target variable

Our analysis

Our target is `ihpEligible`, which covers eligibility for housing and other-needs awards. Whether or not FEMA gives a household aid, yes or no, is the answer our model tries to predict. It is balanced, with 98,648 eligible (50.7%) and 95,834 not eligible (49.3%) (Figure D.1).

About half got aid and half didn't, so the accuracy is a fair score and no SMOTE is required for this dataset. Each piece of the data we train and test on is stratified, so it keeps that same 50.7 / 49.3 mix.

AI-assisted analysis

`ihpEligible` is True when FEMA gave the household money for housing or other needs. Stratifying means each piece of the data keeps the same mix of yes and no.

3.3 Feature roles and data leakage

We sorted all 100 columns into roles. They are listed in `src/utils/columns.py`, which is the one file the whole team uses for this:

Role	Columns	Meaning
Target	1	<code>ihpEligible</code>
Tier 1: application-time	26	What the applicant reports when signing up
Tier 2: later-stage	14	Learned after FEMA inspects or checks the application
Leakage	43	Results of the decision, so never used
ID / constant	15	ID numbers, or the same on every row
Under review	1	<code>utilitiesOut</code> , held out of both tiers for now

Our analysis

Tier 1 has 26 columns, but only 25 features, because `damagedCity` gets dropped and `appliedDate` turns into `daysSinceLandfall`. Tier 2 is Tier 1 with an additional 14 columns, resulting in 39 features total. We're not using `utilitiesOut` yet. It's almost never blank, but when it is, nearly everyone gets approved. That's suspicious and looks like leakage, as it is blank for only 2.7% of rows, but 99.9% of those rows were approved.

We checked each column one at a time to see how closely it matches the answer (Figure 3.1). The score is called Cramér's V, where 0 is no match and 1 is a perfect match. Three columns that are results of FEMA's decision score very high: `ihpAmount` 1.00, `onaEligible` 0.95, `ineligibleReason` 0.70. However, the best normal column only gets 0.46, so anything scoring near 1 is leakage.

The borderline case is the currentLocation variable. It includes things that happen because of aid ("FEMA-provided unit" is 99% eligible), so it was moved from Tier 1 to Tier 2.

ML takeaway: A score only counts if every feature would exist at the moment you make the prediction. That's why we divided the features into two tiers: Tier 1 at registration, Tier 2 after inspection.

AI-assisted analysis

This kind of check is called a univariate leakage test, which means testing one column at a time. No honest column gets close to 1 by itself, so a column that scores near 1.00 is almost surely part of the answer. Moving currentLocation also made the model more honest.

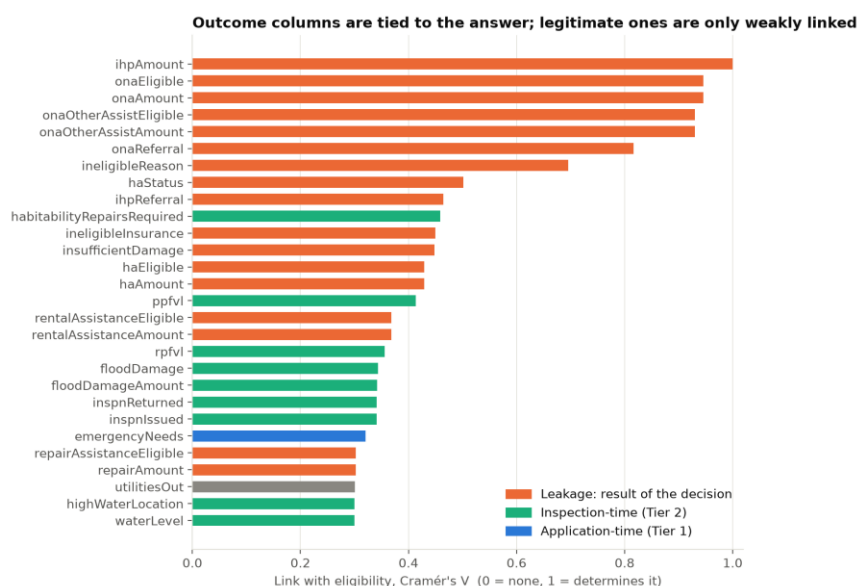


Figure 3.1. Each column's link with eligibility (Cramér's V). Leakage columns sit at the top.

3.4 Summary statistics and distributions

Our analysis

Most applicants were older people in one- or two-person households who owned their homes and applied online (Figure D.2). rpfl reports \$0 in damages for about 83% of people, and the other 17% are widely skewed with large amounts. There are too many ZIP codes in the data, so we log transform the damage skew and frequency-encode the ZIP codes.

AI-assisted analysis

In numbers: 57% of applicants are 50 or older, most households have one or two people, 64% own their home, and 77% signed up online or on the mobile app. There are about 560 different ZIP codes, which is called high cardinality. That is too many to give each one its own column. Frequency encoding swaps each ZIP for how often it shows up, so the model gets one number column instead of hundreds.

ML takeaway: We log transform the damage columns to shrink the skew and add a yes/no "has damage" column. We frequency-encode ZIP codes instead of giving each ZIP its own column.

3.5 Relationships with the target

Our analysis

If the home isn't the applicant's main home, they basically never get approved. Reporting emergency needs makes approval more likely. Having homeowners insurance makes it less likely, and so does higher income. Owning vs. renting doesn't matter.

AI-assisted analysis

The numbers behind this (Figure 3.2): homes that aren't the main home are approved about 0.4% of the time. Reporting emergency needs raises approval from 34% to 66%. Homeowners insurance lowers it from 55% to 47%, since FEMA doesn't pay for damage that insurance already covers. Approval goes from 61% for incomes under \$15k down to 45% above \$175k, but the \$0-income group is the lowest at 42%. Owners and renters are almost the same (51% vs. 50%). At Tier 2, people whose inspection was finished were approved 73% of the time, compared to 38% for everyone else. Approval also runs from 43% to 62% across the 22 biggest ZIP codes (Figures D.3 and D.4).

ML takeaway: Some of the strongest features work like yes/no rules instead of slow trends, like the main-home rule and the \$0-income group. So tree-based models like decision trees and random forests should fit this data well. Income stays a category with no order, because the \$0 group breaks the usual "more income, less aid" pattern.

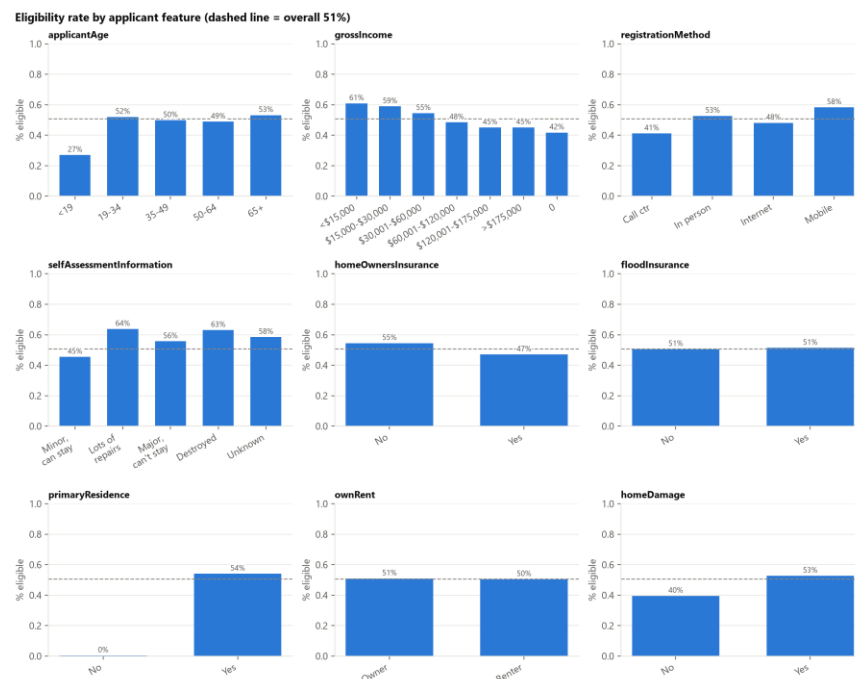


Figure 3.2. Eligibility rate within each category of key features.

3.6 Missing values

Column	Missing	Likely reason it's blank
renterDamageLevel	96%	Only applies to renters
highWaterLocation	88%	Only applies where there was a flood mark
shelterNeed	82%	Almost always "yes" when filled in, so a blank likely means not reported
habitabilityRepairsRequired	73%	Not filled in for most applicants (reason not confirmed)
foodNeed	52%	Almost always "yes" when filled in, so a blank likely means not reported
selfAssessmentInformation	15%	Applicant's own damage rating, left blank by some

Our analysis

When foodNeed is blank, it means the applicant just didn't say they need food. Since the blank tells us something, we shouldn't fill it in with the average.

AI-assisted analysis

Nineteen columns have missing values. Eleven of those are columns we could use, and six are more than 10% missing (Figure D.7). Some blanks are there because the question doesn't apply, like renterDamageLevel for people who aren't renters. So the data is not missing completely at random.

ML takeaway: Filling the blanks with an average would throw away what the blank tells us. So we add a yes/no "was this blank?" column for these features, and fill any other gaps using only the training data.

3.7 Correlation analysis

Our analysis

Not a single column predicts the answer confidently alone, so the model will probably need several columns working together, which will be tested in Milestone 2.

AI-assisted analysis

The strongest single correlation with the target is only about 0.36 (Figure D.5). Some features are almost copies of each other (Figure D.6): inspection issued and inspection completed (1.00), verified home damage and flood damage amount (0.97), and reported damage and home damage (0.83).

ML takeaway: Features that are almost copies of each other are called multicollinearity. We drop one from each pair for linear models like logistic regression. Tree-based models can keep both.

3.8 Data quality assessment

Check	What we found	What we do
Constant columns	14 columns have the same value on every row	Drop them
Duplicates	407 rows (0.21%) are the same in every column except the ID	Keep them
Location errors	6,357 rows (3.3%) have a census block outside Lee County (2,707 are NO_INTERSECT). Only 17 have a ZIP outside Florida	Keep them, with NO_INTERSECT as its own category
Hand-typed city names	Many spellings (FT MYERS, FT MYERS BCH)	Drop the city column
Impossible values	A flood depth of 960 inches (80 feet)	Cap it at a sensible maximum
Dates	Only 2 applications dated before the disaster was declared	Keep them

Our analysis

A flood depth of 960 inches (80 feet) has to be a typo, so we cap it instead of letting one bad row throw off the data.

AI-assisted analysis

The data quality is good overall. Every problem has a fix, and the fixes carry into the Data Preparation Plan (Section 4).

3.9 Challenges and limitations

Our analysis

Our data is only Hurricane Ian in Lee County, so a model trained on it won't necessarily work as well for a hurricane somewhere else, like Texas. None of these issues bring the project to a halt, but it does affect how we build/score the model. Sections 4 and 5 explain how we deal with them.

AI-assisted analysis

Challenge	Why it matters for the model
Data leakage	Scores look perfect but mean nothing if we don't control it
One storm, one county	The model may not work for other disasters (generalization)
Valid registrations only	We can't say anything about applications that never made it in (selection bias)
"Not eligible" covers many reasons	One "no" can mean very different things (label noise)
Heavy skew and blanks that mean something	Needs log transforms and "was this blank?" columns
Too many ZIP codes and census blocks	Encoding them can overfit if it isn't done inside the cross-validation folds

Near-copy features	A problem for linear models (multicollinearity)
Approval partly follows FEMA's own rules	A high score doesn't prove the model understands who needs help

4. Data Preparation Plan

4.1 Data cleaning strategy

Our analysis

We drop the leakage columns because they are results of FEMA's decision, so they would make our model's accuracy look better than it really is.

AI-assisted analysis

Problem (from Section 3)	What we do
Leakage, ID and constant columns (58 columns)	Drop them (listed in Section 3.3)
utilitiesOut (under review)	Leave it out of both tiers until FEMA confirms when it is filled in (Section 3.6)
damagedCity (typed by hand)	Drop it and use ZIP and census block instead
Yes/No columns stored as text with blanks	Change to 1 and 0, and keep blanks as blank (not 0)
Ordered ranges (applicantAge, household counts)	Turn into numbers in order, called ordinal encoding (>5 becomes 6)
Income bracket	Keep as a category with no order (the "\$0" group breaks the order)
Missing values	Don't fill them in during cleaning. Add "was this blank?" columns, then fill with the median for linear models. Boosting models can handle blanks themselves. Anything learned comes from the training data only
Impossible values (waterLevel = 960 in)	Cap at a high percentile
Blocks outside the county	Keep, with NO_INTERSECT as its own category
Rows that look like duplicates	Keep (see Section 3.8)

4.2 Feature engineering

Our analysis

Filling the blanks with the average would hide the fact that the answer was blank, and the blank itself tells us something. So we add a "yes/no" column that says whether it was blank.

AI-assisted analysis

- daysSinceLandfall, made from the application date (already built).
- Log transform ($\log_{1p}$) of the skewed dollar and damage columns, plus yes/no "has damage" columns.
- "Was this blank?" columns for the features where a blank means something.

- ZIP and census block: frequency encoding first. We will also try target encoding, done inside the cross-validation folds.
- Group rare categories together, and drop one of each near-copy pair for linear models.
- Maybe combine features, like emergency needs together with primary residence (an interaction feature), to test in Milestone 2.

4.3 Data transformation

Our analysis

The model only learns from the training piece because if it knows what to expect, it defeats the whole purpose.

AI-assisted analysis

Scaling (putting numbers on the same scale) and one-hot encoding (one yes/no column per category) are only used for the models that need them, like logistic regression and distance-based methods. Tree-based models use the data as it is. Anything that learns from the data, like filling blanks, scaling, and encoding, is learned from the training piece only, inside a scikit-learn Pipeline. This stops a second kind of leakage, where the test data affects training.

4.4 Train / validation / test strategy

Our analysis

Because that is our final score for the model. If we used the test set more than once, we would start adjusting the model to it and the score wouldn't be honest.

AI-assisted analysis

- Stratified split: 60% train, 20% validation, 20% test, with random seed 42 so everyone gets the same split. Each piece keeps the same 50.7% / 49.3% mix. This is done in `split_data()`.
- The test set is used once, at the very end of Milestone 2.
- Cross-validation (Section 5.3) uses the train and validation pieces together (80%). The 20% test set stays untouched.
- Time check: applications came in over time, so we will also train on early applicants and test on later ones to see if the model still holds up.
- We can't link applications from the same household, so we list this as a limit.

5. Modeling Approach

5.1 Algorithm selection and justification

Our analysis

We start with logistic regression because it's a very simple and quick model, making it a good baseline for testing other models. Some of our strongest features work like yes/no rules, like whether the home is the primary residence, and that's exactly what tree models are good at.

AI-assisted analysis

Model	Why it fits this data
Logistic regression	Simple, fast, and easy to explain. The baseline to beat
Decision tree	Makes rules a person can read, and FEMA's process works like rules
Random forest	Handles skew, mixed data types, and outliers. Using many trees makes it less likely to overfit than one tree
Gradient boosting	Can pick up features working together (Section 3.7) and the yes/no cutoffs (Section 3.5)

The baselines have not been re-run on the verified data yet. The earlier results came from `src/models/baseline.py` on an older download, and the EDA notebook doesn't produce them anymore. We will re-run the baselines on the verified data in Milestone 2 and report ROC-AUC, accuracy, and F1 for Tier 1 and Tier 2 then.

Reference	ROC-AUC	Accuracy	F1
Always guess the majority class (50.7% eligible)	0.500	0.507	0.000

5.2 Evaluation metrics

Our analysis

Telling someone they'll get aid when they won't is arguably worse because it delays the amount of preparation that they'll have to do as a result of waiting for support from FEMA.

AI-assisted analysis

Metric	Why we use it
ROC-AUC (main)	Shows how well the model ranks eligible above not eligible, no matter where the cutoff is. A good fit because the classes are balanced
Precision, Recall, F1	A wrong "not eligible" and a wrong "eligible" cost different things in real life, so we report both and pick the cutoff on purpose
Accuracy	Fair here because the classes are balanced
Confusion matrix	Shows exactly where the mistakes are
Group error rates (age, income)	Fairness check (target in Section 2.2)

5.3 Cross-validation and hyperparameter tuning

Our analysis

Testing five times lets us see how much the model's score changes with each run, so one lucky or unlucky split doesn't fool us.

AI-assisted analysis

We use five-fold stratified cross-validation on the train and validation data. The search for the best settings (randomized hyperparameter search) runs inside the folds. Every model uses the same folds so the comparison is fair. The test set is used once, for the final score.

5.4 Expected challenges and mitigation

AI-assisted analysis

Challenge	What we do about it
Hidden leakage when scores get high	Keep one roles file for the whole team. For every final feature set, drop a group of features and see what changes (an ablation study). The EDA's location check is how currentLocation got moved out of Tier 1
Too many ZIP codes and census blocks	Do frequency and target encoding inside the folds, and compare with and without
Skewed damage columns that are mostly zero	Log transform plus yes/no columns, and lean on tree models
Model just re-learns FEMA's rules	Say so honestly, and look closely at cases where the model disagrees with the outcome
Fairness across age, income, and place	Report error rates by group and look into any gap over 5 points
Results only apply to one storm	Say what the scope is, and run the time check

6. Project Timeline

Dates	Work	Owner
Sep 14–27	Project setup, data download, first EDA and report draft	All
Sep 28–Oct 6	Section writing; EDA notebook rebuilt on the verified data	All
Oct 7	Milestone 1 due	All
Oct 8–21	Finish the data preparation code and the notebooks/preprocessing notebook	All
Oct 22–31	Re-run baselines, train all models, cross-validate, and tune settings	All
Nov 1–10	Evaluation: compare models, see which features matter most, check for leakage, check fairness	All
Nov 11–18	Write the final report, build the presentation, and rehearse	All
Nov 19	Team deadline: code runs start to finish, PDF made	All
Nov 21	Milestone 2 due	All
Nov 23 / Dec 2	Presentation (15 min + Q&A)	All

Team responsibilities for Milestone 2. Daniel Ortiz leads data preparation and model training. Anthony Stoneking leads the data analysis updates, leakage checks, and model evaluation. Nandor Laar leads the business value and fairness review and the final report. Everyone contributes to every phase and attends the presentation.

Risks and contingencies

Risk	Contingency
A teammate is unavailable	Everything is in Git with comments in the code, so someone else can pick it up. Weekly check-ins
Scores drop below the baseline after removing leakage columns	Report it honestly. Removing leakage is a finding too
Conflicts when two people edit the same notebook	One person per notebook at a time, small commits, pull before starting
Running out of time before Nov 21	Lock the list of models by Oct 31 and drop the optional ones first

Appendices

- A. Data dictionary (docs/data_dictionary.md)
- B. EDA notebook export (reports/milestone_1/eda_notebook.html)
- C. AI usage log (docs/AI_USAGE.md)
- D. Additional figures (reports/figures/)

Appendix D. Additional figures for Section 3

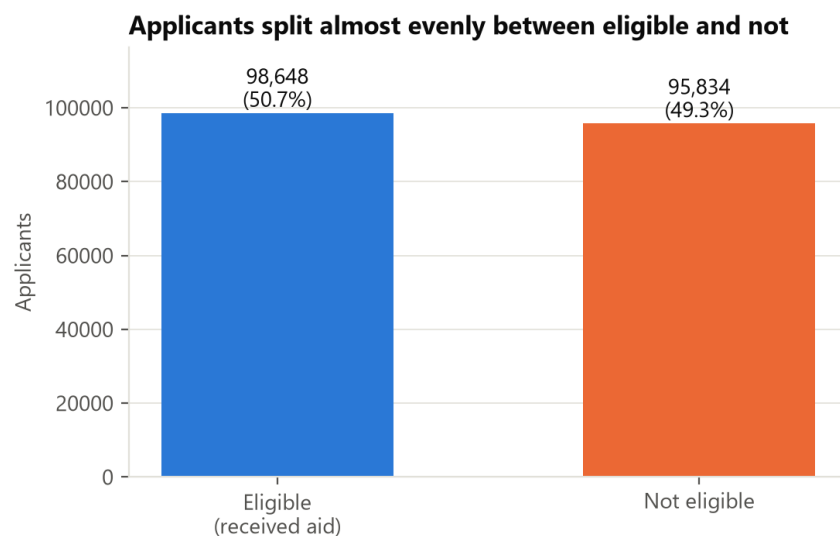


Figure D.1. Class balance of ihpEligible.

Applicants skew older, small households, homeowners, and internet/mobile registration

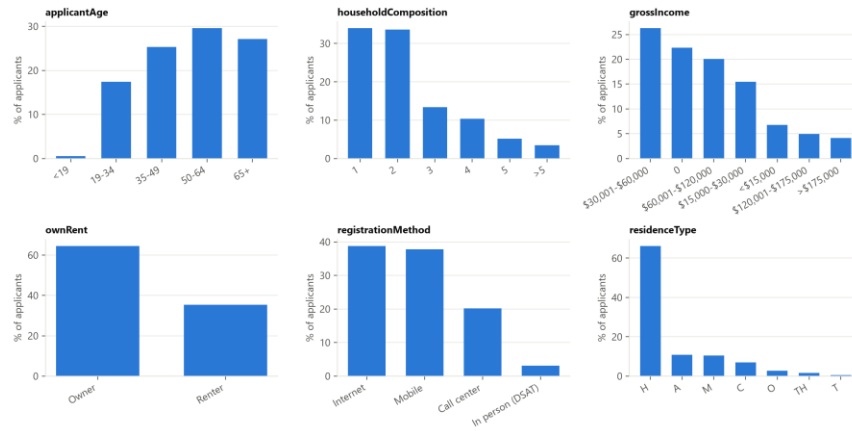


Figure D.2. Age, household size, ownership, and registration method.

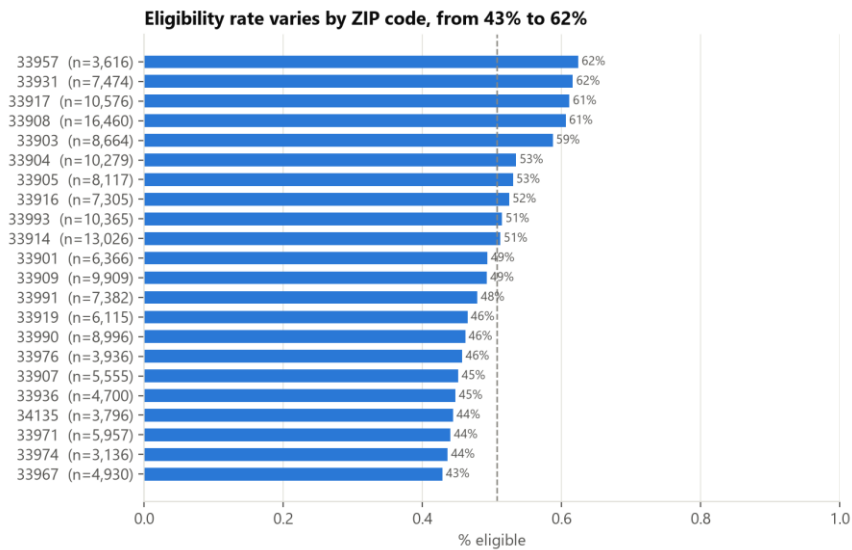


Figure D.3. Eligibility across the 22 largest ZIP codes.

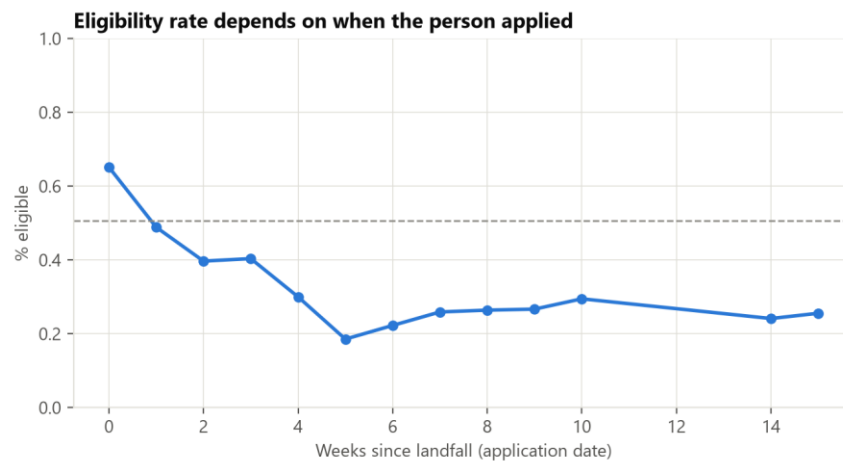


Figure D.4. Eligibility by week of application.

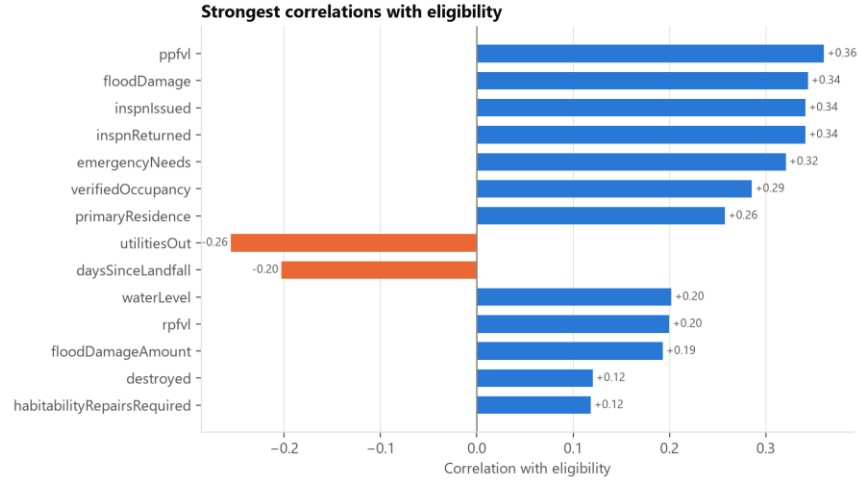


Figure D.5. Correlation of each numeric feature with the target.

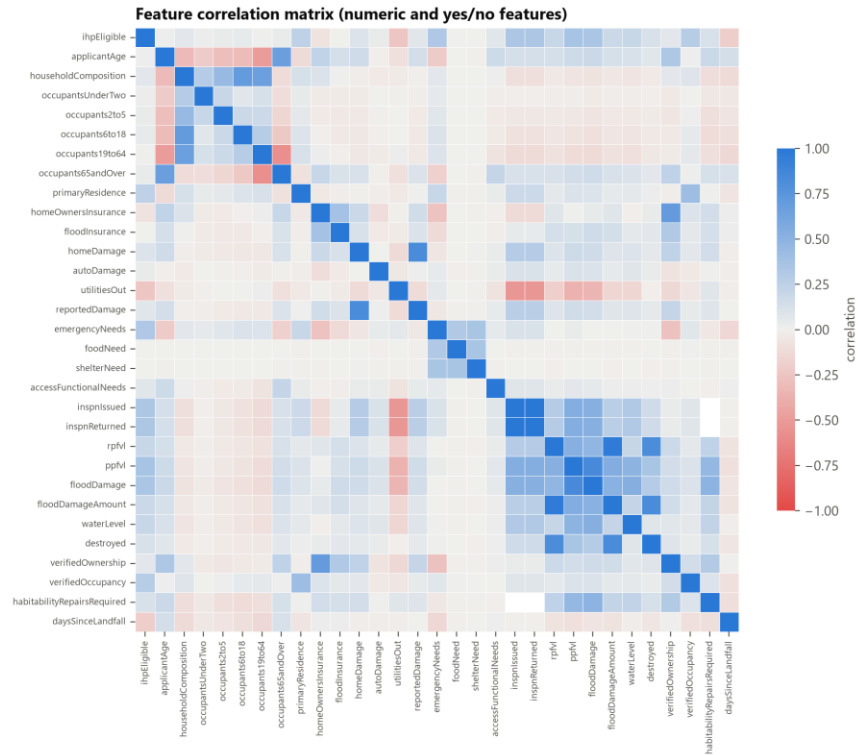


Figure D.6. Feature-to-feature correlations.

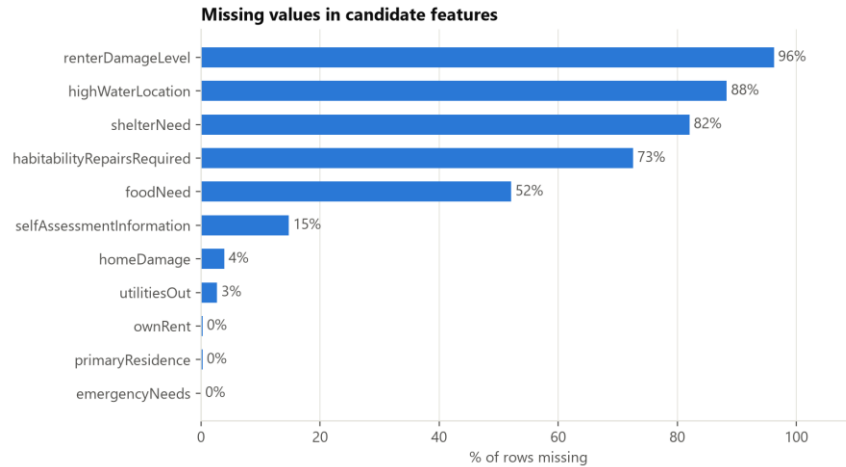


Figure D.7. Share of missing values per column.

References

FEMA. *OpenFEMA Dataset: Individuals and Households Program – Valid Registrations v2*.

<https://www.fema.gov/openfema-data-page/individuals-and-households-program-valid-registrations-v2>

FEMA. *Serious Needs Assistance* (fact sheet). <https://www.fema.gov/fact-sheet/serious-needs-assistance-0>

AI-use disclosure

Portions of the code, analysis, and this draft were produced with Claude Code (Anthropic) and reviewed by the team. See docs/AI_USAGE.md for the full log.